Topic : Terraform
Date : 26/10/2026

Infrastructure as Code

Multi-Cloud Ready

Modular & Reusable

Declarative Language (HCL)

Safe & Reliable

TERRAFORM

Build • Change • Manage Your Infrastructure

What You Will Learn

- Write Infrastructure as Code
- Manage Resources Across Clouds
- Automate, Version & Collaborate
- Secure, Scalable & Cost Effective
- Consistent & Repeatable

Terraform Workflow

Write (Code) → Plan (Preview) → Apply (Provision) → Manage (Operate) → Destroy (Clean Up)

TERRAFORM

Topic : Terraform
Date : 26/10/2026

T1. INTRODUCTION TO TERRAFORM & INFRASTRUCTURE AS CODE

1. What is Terraform ?

Terraform is an **open-source** Infrastructure as Code (IaC) tool created by **HashiCorp.** It allows you to **define, provision and manage** infrastructure across multiple cloud providers in a safe, efficient and repeatable way.

HashiCorp Terraform

Key Highlights

- ✔ Infrastructure as Code (IaC)
- ✔ Multi-Cloud Support
- ✔ Declarative Language (HCL)
- ✔ Plan, Preview & Apply Changes
- ✔ State Management
- ✔ Reusable & Modular
- ✔ Automation & Consistency

2. What is Infrastructure as Code (IaC) ?

Infrastructure as Code (IaC) means managing and provisioning infrastructure using **code** instead of manual processes. You write configuration files that describe the infrastructure, and Terraform takes care of creating and managing it.



3. Why Use Terraform ?

- ✔ Automate infrastructure provisioning
- ✔ Reduce manual errors
- ✔ Ensure consistency across environments
- ✔ Version control & collaboration
- ✔ Reusable & modular configurations
- ✔ Faster deployment & easy scaling
- ✔ Supports multiple cloud providers

4. How Terraform Works (High Level)



Write (Code)	Plan (Preview)	Apply (Provision)	Manage (Operate)	Destroy (Clean up)
Define configuration in HCL files	Shows changes before apply	Creates / Updates infrastructure	Monitor and manage resources	Removes resources when not needed

5. Supported Cloud Providers (Examples)

AWS | Azure | Google Cloud | Kubernetes | On-Premises | Many More...

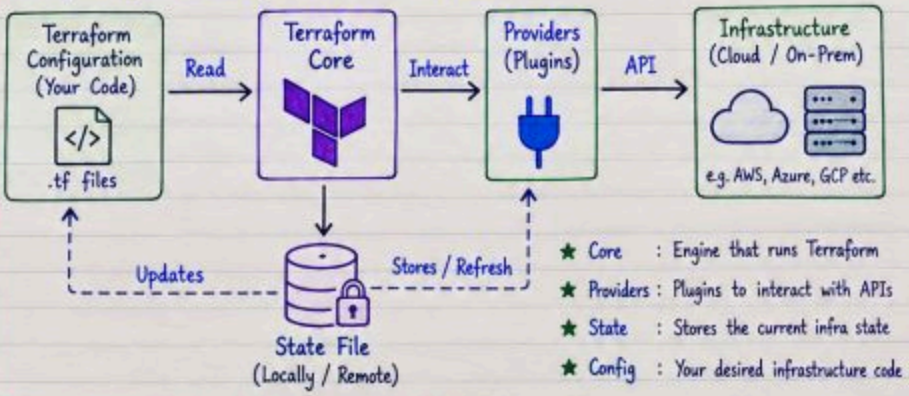
TERRAFORM

Topic : Terraform
Date : 26/10/2026

T2. TERRAFORM ARCHITECTURE & WORKFLOW

1. Terraform Architecture

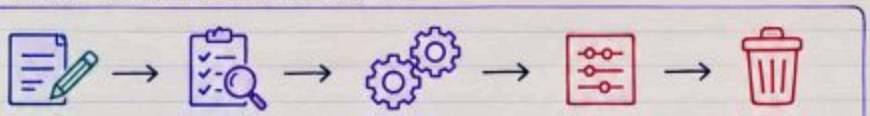
Terraform follows a simple client-server architecture.



- ★ Core : Engine that runs Terraform
- ★ Providers : Plugins to interact with APIs
- ★ State : Stores the current infra state
- ★ Config : Your desired infrastructure code

2. Terraform Workflow (High Level)

Terraform follows a declarative workflow.



1. **Write** — Write configuration code in .tf files
2. **Plan** — Preview the changes Terraform will make
3. **Apply** — Apply the changes to real infrastructure
4. **Manage** — Manage and update the infrastructure
5. **Destroy** — Destroy (clean up) infrastructure

3. Key Components

- Configuration Files (.tf) : Define desired state
- Providers : Connect to APIs (AWS, Azure, GCP, etc.)
- Resources : Infrastructure components (EC2, VPC, S3, etc.)
- State File : Tracks current state of infrastructure
- Variables : Make configuration dynamic & reusable
- Outputs : Show important information after apply

4. State File Role

- Acts as Terraform's source of truth.
- Maps real infrastructure to your configuration.
- Stores metadata and state (e.g., resource IDs, dependencies).
- Helps in detecting changes and ensures consistency.
- Locking prevents concurrent changes & corruption.



TERRAFORM

Topic : Terraform
Date : 26/10/2026

T3. TERRAFORM INSTALLATION & SETUP

1. Prerequisites

- ✓ Operating System : Windows / macOS / Linux
- ✓ Internet Connection : Required to download Terraform
- ✓ Admin / sudo access : Required for installation
- ✓ A Text Editor or IDE : VS Code (Recommended)

2. Install Terraform

Windows

1. Download Terraform zip from official website
   `https://developer.hashicorp.com/terraform/downloads`
2. Extract the zip file
3. Move terraform.exe to a folder (e.g. C:\terraform)
4. Add the folder to System Environment PATH.

```
# Check in CMD
terraform -version
```

macOS

1. Using Homebrew (Recommended)
   `brew install terraform`

— OR —

2. Download the .zip from official website
3. Unzip and move to a directory (e.g. /usr/local/bin)

```
# Check in Terminal
terraform -version
```

Linux (Ubuntu/Debian)

1. Download the official package
   `wget https://releases.hashicorp.com/terraform/latest/terraform_linux_amd64.zip`
2. Unzip the file
   `unzip terraform_*.zip`
3. Move binary to /usr/local/bin
   `sudo mv terraform /usr/local/bin/`
4. Give executable permission
   `sudo chmod +x /usr/local/bin/terraform`

☆ Now Terraform is installed on your system.

3. Verify Installation

Check Terraform version:

```
terraform -version
```

You should see output like this :

```
Terraform v1.7.x
on windows_amd64 (or linux_amd64)
+ provider registry.terraform.io/hashicorp/aws v5.x.x
Your version of Terraform is up to date!
```

4. Setup IDE (VS Code Recommended)

- ✓ Install Visual Studio Code (VS Code)
- ✓ Install HashiCorp Terraform extension
- ✓ Useful extensions :
  - HashiCorp Terraform
  - YAML
- ✓ Create a new folder for your Terraform project.

5. First Project Setup

1. Create Project Folder
   ```
   mkdir my-terraform-project
   cd my-terraform-project
   ```
2. Create a file (main.tf)
   ```
   touch main.tf
   ```
3. Initialize Terraform
   ```
   terraform init
   ```
   (Downloads providers)
4. Validate Configuration
   ```
   terraform validate
   ```
   (Checks configuration)

TERRAFORM

Topic : Terraform
Date : 26/10/2026

T4. TERRAFORM CLI – IMPORTANT COMMANDS

Terraform CLI (Command Line Interface) is used to initialize, plan, apply and manage your infrastructure as code.



No.	Command	Description	Example
1	terraform init	Initializes a working directory. Downloads provider plugins and initializes the backend.	terraform init
2	terraform validate	Validates the configuration files in the working directory.	terraform validate
3	terraform fmt	Rewrites config files to a consistent format.	terraform fmt
4	terraform plan	Creates an execution plan. Shows what changes Terraform will make.	terraform plan
5	terraform apply	Applies the changes required to reach the desired state.	terraform apply
6	terraform destroy	Destroys all managed infrastructure.	terraform destroy
7	terraform show	Shows the current state or a saved plan in human-readable form.	terraform show
8	terraform state list	Lists all resources in the state.	terraform state list
9	terraform state show <res>	Shows details of a specific resource in the state.	terraform state show aws_instance.web
10	terraform taint <res>	Marks a resource for recreation during next apply.	terraform taint aws_instance.web
11	terraform untaint <res>	Removes the taint from a resource.	terraform untaint aws_instance.web
12	terraform import <res> <id>	Imports an existing resource into Terraform state.	terraform import aws_s3_bucket.mybucket my-bucket
13	terraform refresh	Updates the state file with real infrastructure.	terraform refresh
14	terraform output	Shows output values from the root module.	terraform output
15	terraform workspace	Manages workspaces.	terraform workspace list
16	terraform version	Shows Terraform version.	terraform version

Quick Tips

- ✓ Run terraform init before using other commands.
- ✓ Use terraform plan to preview changes.
- ✓ Use meaningful names for resources.
- ✓ Keep your .tf files formatted using terraform fmt.
- ✓ Use workspaces for different environments (dev, staging, prod).

Command Execution Flow



TERRAFORM

Topic : Terraform
Date : 26/10/2026

T5. HCL – TERRAFORM CONFIGURATION LANGUAGE

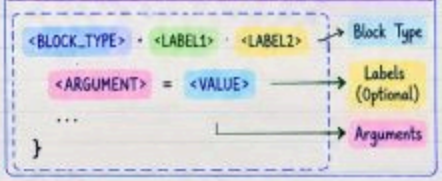
HCL (HashiCorp Configuration Language) is the declarative language used by Terraform to define infrastructure.
It is human readable and designed to describe configuration in blocks.

HCL { }

1. HCL Basics

- ✔ HCL uses blocks to represent configuration.
- ✔ Blocks have labels and arguments.
- ✔ Arguments use key = value pairs.
- ✔ HCL is case sensitive.
- ✔ Comments start with # or //.

2. Basic Structure of an HCL Block



3. Example – Simple EC2 Instance Configuration

```
resource "aws_instance" "example" {
  ami           = "ami-0c55b159cbfafelf0"
  instance_type = "t2.micro"
  key_name      = "my-key"
  tags = {
    Name        = "MyEC2Instance"
    Environment = "Dev"
  }
}
```

- resource : Block type (inbuilt Terraform block)
- aws_instance : First label (Provider + Resource type)
- example : Second label (Logical name)
- Arguments : Key = Value pairs inside the block
- tags : Map of key-value pairs

4. Common HCL Elements

Element	Example	Description
Block	resource "aws_s3_bucket" "b" { ... }	Defines a configuration block
Argument	bucket = "my-bucket"	Key = Value inside a block
String	"hello"	Text value
Number	42	Numeric value
Boolean	true / false	Boolean value
List	["a", "b", "c"]	Ordered collection
Map	{ Name = "Anush" }	Key-value collection

5. Comments in HCL

```
# This is a single line comment
// This is also a single line comment
/*
  This is a
  multi-line
  comment
*/
```

Comments are ignored by Terraform.

6. HCL File Extension

Terraform configuration files use .tf extension.

.tf → main.tf (Configuration File)

7. Key Points

- ✔ HCL is declarative, not procedural.
- ✔ You describe the desired state, Terraform handles the rest.
- ✔ HCL is designed to be simple, readable and maintainable.
- ✔ Configuration is written in .tf files.
- ✔ Supports variables, modules, providers and resources.

TERRAFORM

Topic : Terraform
Date : 26/10/2026

T6. PROVIDERS, RESOURCES & DATA SOURCES

Terraform uses **Providers** to communicate with cloud platforms. You define **Resources** to create infrastructure and use **Data Sources** to fetch information about existing infrastructure.

AWS | Azure | GCP

1. Providers

- ✔ A provider is a plugin that lets Terraform interact with a platform or service.
- ✔ Examples: aws, azurerm, google, random, kubernetes, etc.
- ✔ Configure provider with credentials & region.

Example – AWS Provider

```
provider "aws" {
  region     = "ap-south-1"
  access_key = "AKIAxxxxxxxxxxx"
  secret_key = "xxxxxxxxxxxxxxxxxxxx"
}
```

Note : Credentials can also be loaded from environment variables or IAM roles.

2. Resources

- ✔ Resources are the infrastructure objects managed by Terraform.
- ✔ You define the desired state using resource blocks.
- ✔ Terraform will create, update or delete resources based on configuration.

Example – AWS EC2 Instance (Resource)

```
resource "aws_instance" "web" {
  ami           = "ami-0c55b159cbfafe1f0"
  instance_type = "t2.micro"
  key_name      = "my-key"
  tags = {
    Name        = "MyEC2Instance"
    Environment = "Dev"
  }
}
```

Key Points

- ✔ Resource type: aws_instance
- ✔ Logical name: web
- ✔ Arguments inside the block define the settings.

3. Data Sources

- ✔ Data sources are used to fetch information about existing resources.
- ✔ They do not manage infrastructure.
- ✔ Useful for referencing values that already exist (VPC, AMI, Subnet, etc.).

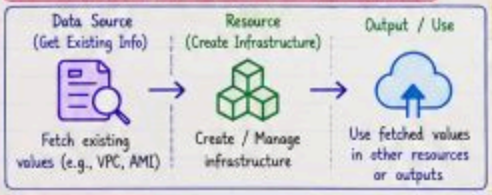
Example – Fetch Existing AMI (Data Source)

```
data "aws_ami" "ubuntu" {
  most_recent = true
  filter {
    name   = "name"
    values = ["ubuntu/images/hvm-ssd/ubuntu-focal-20.04-amd64-server-*"]
  }
  owners = ["099720109477"]  # Canonical
}
```

Key Points

- ✔ Used with data keyword.
- ✔ Helps to read information.
- ✔ Result can be used in resources.

4. Using Resources & Data Sources Together



Example

```
data "aws_vpc" "default" {
  default = true
}

resource "aws_subnet" "public" {
  vpc_id            = data.aws_vpc.default.id  # using data source
  cidr_block        = "10.0.1.0/24"
  availability_zone = "ap-south-1a"
}
```

5. Summary

Concept	Purpose	Type	Example
Provider	Connects Terraform to a platform	Configuration Block	provider "aws" { ... }
Resource	Create / Manage infrastructure	Resource Block	resource "aws_instance" "web" { ... }
Data Source	Read / Fetch existing information	Data Block	data "aws_vpc" "default" { ... }

Page - 7

TERRAFORM

Topic : Terraform
Date : 26/10/2026

T7. VARIABLES, LOCALS & OUTPUTS

Terraform provides **variables** to make configurations dynamic, **locals** to store intermediate values and **outputs** to display important information.

Variables | Locals | Outputs

1. VARIABLES

- ✓ Variables make your code dynamic and reusable.
- ✓ Defined using the `variable` block.
- ✓ Values are provided in `.tfvars` file or via CLI.
- ✓ Used with `var.variable_name`.

Example - variables.tf (HCL)

```
variable "region" {
  description = "AWS region to deploy resources"
  type        = string
  default     = "ap-south-1"
}

variable "instance_type" {
  description = "EC2 instance type"
  type        = string
  default     = "t2.micro"
}
```

Example - terraform.tfvars (TFVARS)

```
region        = "ap-south-1"
instance_type = "t2.micro"
```

Key Points

- ✓ Variables improve reusability.
- ✓ Provide default values if possible.
- ✓ Sensitive data can be marked as `sensitive = true`.

2. LOCALS

- ✓ Locals store intermediate values.
- ✓ Defined using the `locals` block.
- ✓ Useful to avoid repetition in values or expressions.

Example - locals.tf (HCL)

```
locals {
  name_prefix = "demo-${var.environment}"
  common_tags = {
    Project     = "TerraformDemo"
    Environment = var.environment
    Owner       = "DevOps"
  }
}
```

Example - Usage in resource (HCL)

```
resource "aws_instance" "web" {
  ami           = "ami-0c5b159cbfafe1f0"
  instance_type = var.instance_type
  tags          = local.common_tags
}
```

Key Points

- ✓ Locals reduce repeating values.
- ✓ Evaluated once and used multiple times.

3. OUTPUTS

- ✓ Outputs display important information after apply.
- ✓ Defined using the `output` block.
- ✓ Useful to show values like IDs, IPs, DNS names, etc.

Example - outputs.tf (HCL)

```
output "instance_id" {
  description = "ID of the EC2 instance"
  value       = aws_instance.web.id
}

output "public_ip" {
  description = "Public IP of the instance"
  value       = aws_instance.web.public_ip
}
```

Example - Output after apply (Output)

```
Outputs:
instance_id = "i-0a1b2c3d4e5f6g7h8"
public_ip   = "54.123.45.67"
```

Key Points

- ✓ Outputs are shown at the end of apply.
- ✓ Can be used by other modules via `module.<name>.output_<name>`.

4. VARIABLES vs LOCALS vs OUTPUTS

Feature	Variables	Locals	Outputs
Purpose	Input values to configure code	Store intermediate values	Show important information
Defined Using	variable block	locals block	output block
Provide By	User (.tfvars / CLI / env)	Defined in code	Defined in code
Change During Run	Yes (by user)	No (fixed in code)	No (read-only result)
Used For	Customization	Code reusability & simplification	Information / Integration
Example	var.region	local.common_tags	output.instance_id

TERRAFORM

Topic : Terraform
Date : 26/10/2026

T8. TERRAFORM PLAN, APPLY & DESTROY

Terraform follows a simple workflow: Plan → Apply → Destroy
These commands help you preview, create/update and remove infrastructure.



1. TERRAFORM PLAN

- ✓ Plans your changes.
- ✓ Shows what resources will be created, updated or destroyed.
- ✓ Does not make any real changes.
- ✓ Generates an execution plan.

Example

```
$ terraform plan
# Output (summary)
+ 3 to add
~ 1 to change
- 0 to destroy
Plan: 3 to add, 1 to change, 0 to destroy.
```

Key Points

- ✓ Safe to run.
- ✓ Preview before making changes.
- ✓ Use -out to save plan to a file.

Cmd: terraform plan

2. TERRAFORM APPLY

- ✓ Applies the changes required to reach the desired state.
- ✓ Creates, updates or deletes resources.
- ✓ Asks for confirmation before making changes.

Example

```
$ terraform apply
Terraform will perform the following actions:
+ aws_instance.web will be created
~ aws_s3_bucket.data will be updated
Plan: 2 to add, 1 to change, 0 to destroy.
Do you want to perform these actions?
Enter a value: yes
Apply complete! Resources: 2 added, 1 changed,
0 destroyed.
```

Key Points

- ✓ Actually makes the changes.
- ✓ Use -auto-approve in non-interactive mode.
- ✓ Always run plan before apply.

Cmd: terraform apply

3. TERRAFORM DESTROY

- ✓ Destroys all resources managed by Terraform.
- ✓ Removes infrastructure.
- ✓ Asks for confirmation before destroying.

Example

```
$ terraform destroy
Terraform will perform the following actions:
- aws_instance.web will be destroyed
- aws_s3_bucket.data will be destroyed
Plan: 0 to add, 0 to change, 2 to destroy.
Do you really want to destroy all resources?
Enter a value: yes
Destroy complete! Resources: 2 destroyed.
```

Key Points

- ✓ Deletes all managed resources.
- ✓ Use with caution.
- ✓ Always backup important data.

Cmd: terraform destroy

Workflow Summary



1. Plan
Preview changes
(Safe)



2. Apply
Implement changes
(Creates / Updates)



3. Destroy
Remove infrastructure
(Use with caution)